

1-Portada

Nombre del proyecto: MoneyTrack: Aplicación móvil para el control de gastos personales.

Equipo: Arnold Mejía Tineo.

Asignatura: Seminario de Proyecto II.

Clave: ISW-411.

Carrera: Ingeniería de Software.

Universidad: Universidad Abierta para Adultos (UAPA).

Fecha: 15 de julio del 2026.

2-Descripción del problema

Actualmente, muchas personas presentan dificultades para administrar correctamente sus finanzas personales debido a la falta de herramientas sencillas que les permitan registrar y consultar sus gastos diarios. En muchos casos, los gastos se recuerdan de manera aproximada o se anotan en diferentes medios, lo que dificulta llevar un control preciso del dinero utilizado y tomar mejores decisiones financieras.

Con el propósito de atender esta necesidad surge MoneyTrack, una aplicación móvil desarrollada para dispositivos Android que permite registrar, consultar, actualizar y eliminar gastos personales utilizando una base de datos SQLite almacenada localmente en el dispositivo. La aplicación ofrece una solución práctica y de fácil uso para organizar la información financiera sin depender de conexión a Internet ni de servicios externos.

3-Objetivos y alcance

Objetivo general

Desarrollar una aplicación móvil para Android que permita a los usuarios registrar y administrar sus gastos personales mediante almacenamiento local en SQLite, ofreciendo una herramienta sencilla para mejorar el control de sus finanzas.

Objetivos específicos

- Implementar un sistema de registro e inicio de sesión para los usuarios.
- Desarrollar un CRUD completo para la gestión de gastos personales.
- Almacenar la información utilizando una base de datos SQLite.
- Mostrar un Dashboard con información resumida de los gastos registrados.
- Aplicar validaciones en los formularios para garantizar la integridad de los datos.

Alcance

La aplicación permite registrar usuarios, iniciar sesión, registrar gastos, consultar la lista de gastos, actualizar registros existentes, eliminar información y visualizar un resumen financiero desde el Dashboard. Todo el funcionamiento se realiza utilizando almacenamiento local mediante SQLite.

Funcionalidades excluidas

Por limitaciones de tiempo y por el alcance definido para esta primera versión, no se implementaron funcionalidades como sincronización en la nube, generación de reportes en PDF, recuperación de contraseña, gráficos estadísticos, control de presupuestos, exportación de datos o sincronización entre dispositivos.

4-Justificación e implicaciones socioeconómicas

El desarrollo de MoneyTrack responde a la necesidad de contar con una herramienta sencilla que facilite el registro y control de los gastos personales desde un dispositivo móvil. La aplicación permite que cualquier usuario pueda organizar mejor su información financiera y conocer cuánto dinero ha gastado, favoreciendo una mejor planificación económica.

Desde el punto de vista educativo, el proyecto permitió aplicar los conocimientos adquiridos durante la carrera en áreas como desarrollo de aplicaciones móviles, programación orientada a objetos, bases de datos SQLite y arquitectura de software. A nivel social, la aplicación promueve hábitos de organización financiera que pueden contribuir a una administración más responsable de los recursos económicos personales.

5-Factibilidad

La factibilidad del proyecto **MoneyTrack** fue analizada considerando los aspectos técnicos, el tiempo disponible para el desarrollo y los recursos necesarios para su implementación. Este análisis permitió determinar que el proyecto era viable dentro de las condiciones establecidas para la asignatura, ya que las tecnologías seleccionadas eran conocidas por el desarrollador, los recursos requeridos estaban disponibles y el cronograma de trabajo era suficiente para cumplir con los objetivos planteados.

Factibilidad técnica

Desde el punto de vista técnico, el proyecto fue completamente viable debido al uso de tecnologías ampliamente utilizadas para el desarrollo de aplicaciones móviles Android. Se seleccionó **Android Studio** como entorno de desarrollo por ser la herramienta oficial para la creación de aplicaciones Android y ofrecer integración con el SDK, el emulador y las herramientas de depuración.

Como lenguaje de programación se utilizó **Java**, ya que fue el lenguaje trabajado durante la asignatura y proporciona una integración completa con Android Studio, además de una amplia documentación y recursos de aprendizaje.

Para el almacenamiento de la información se implementó **SQLite**, una base de datos local integrada en Android que permite guardar la información de manera permanente sin necesidad de utilizar un servidor o conexión a Internet. Esta elección permitió desarrollar una aplicación funcional, estable y adecuada para el alcance definido del proyecto.

La arquitectura **Modelo-Vista-Controlador (MVC)** facilitó la organización del código, separando la lógica de negocio, la interfaz de usuario y el acceso a los datos, lo que mejoró la mantenibilidad y comprensión del sistema.

Factibilidad de tiempo

El desarrollo del proyecto fue planificado de acuerdo con el cronograma establecido para la asignatura **Seminario de Proyecto II**, distribuyendo el trabajo en diferentes etapas que abarcaron el análisis del problema, la elaboración de la documentación, el diseño de la base de datos, el desarrollo de la aplicación, las pruebas funcionales y la documentación final.

Cada entrega parcial permitió avanzar de forma progresiva hasta obtener una aplicación completamente funcional. La planificación del tiempo hizo posible implementar las funcionalidades principales sin afectar el cumplimiento de las fechas de entrega establecidas por la asignatura.

Factibilidad de recursos

El proyecto no requirió infraestructura especializada ni herramientas de alto costo para su desarrollo. Los recursos utilizados fueron una computadora personal con capacidad para ejecutar Android Studio, un dispositivo móvil Android y un emulador para realizar las pruebas de funcionamiento.

Asimismo, se utilizaron herramientas gratuitas como **GitHub** para el control de versiones y respaldo del código fuente, además de la documentación oficial de Android Developers, Java y SQLite como apoyo durante el proceso de desarrollo.

Al tratarse de una aplicación con almacenamiento local, no fue necesario contratar servicios de alojamiento, servidores web o bases de datos remotas, lo que redujo significativamente los costos de implementación y permitió desarrollar el proyecto utilizando únicamente recursos disponibles para el estudiante.

6-Tecnologías utilizadas

Para el desarrollo de **MoneyTrack** se seleccionó un conjunto de tecnologías que permitieron construir una aplicación móvil funcional, estable y acorde con los requisitos establecidos para el proyecto final. La elección de cada herramienta se realizó considerando su compatibilidad con Android, facilidad de uso, disponibilidad de documentación y adecuación al alcance definido para esta primera versión.

Android Studio

Android Studio fue utilizado como el entorno de desarrollo integrado (IDE) principal para la construcción de la aplicación. Esta herramienta fue seleccionada por ser el entorno oficial recomendado por Google para el desarrollo de aplicaciones Android, ofreciendo integración completa con el Android SDK, el emulador de dispositivos, herramientas de depuración y administración de proyectos.

Además, Android Studio facilita el diseño de interfaces, la compilación del proyecto, la detección de errores durante el desarrollo y la generación del archivo APK necesario para la instalación de la aplicación. Su integración con Gradle permitió administrar las dependencias y mantener una estructura organizada del proyecto.

Java

El lenguaje de programación utilizado fue **Java**, seleccionado por ser uno de los lenguajes principales para el desarrollo de aplicaciones Android y por formar parte de los contenidos trabajados durante la asignatura. Java permitió implementar toda la lógica de negocio del sistema, incluyendo el registro e inicio de sesión de usuarios, la gestión de gastos, las validaciones de formularios y la comunicación con la base de datos SQLite.

Otra razón para elegir Java fue su estabilidad, amplia documentación oficial y gran comunidad de desarrolladores, lo que facilitó la resolución de problemas durante el proceso de implementación.

SQLite

Como sistema de almacenamiento de datos se utilizó **SQLite**, una base de datos relacional integrada de forma nativa en Android. Su elección respondió a la necesidad de disponer de una solución ligera, rápida y que funcionara sin conexión a Internet.

SQLite permitió almacenar de forma persistente la información de los usuarios y los gastos registrados, implementando las operaciones de crear, consultar, actualizar y eliminar registros (CRUD). Al no depender de servidores externos, simplificó el desarrollo y cumplió con los objetivos planteados para esta versión del proyecto.

XML

El diseño de las interfaces gráficas fue desarrollado utilizando **XML (Extensible Markup Language)**, lenguaje empleado por Android para definir la estructura visual de las pantallas. Mediante XML se construyeron las vistas correspondientes al inicio de sesión, registro de usuario, dashboard, registro de gastos y lista de gastos.

La utilización de XML permitió mantener separada la interfaz gráfica de la lógica de programación implementada en Java, favoreciendo una mejor organización del proyecto y facilitando futuras modificaciones en el diseño sin afectar el funcionamiento del sistema.

Git y GitHub

Durante el desarrollo del proyecto se utilizó **Git** como sistema de control de versiones y **GitHub** como plataforma para almacenar el repositorio del proyecto. Estas herramientas permitieron llevar un registro de los cambios realizados, mantener copias de seguridad del código y documentar la evolución del desarrollo mediante commits.

El uso de GitHub también facilitó la entrega del proyecto, ya que permitió disponer del código fuente organizado y accesible, además de incluir el archivo APK y la documentación requerida para la evaluación.

Arquitectura MVC

Como patrón de arquitectura se implementó el modelo **MVC (Modelo–Vista–Controlador)**. Esta arquitectura permitió separar la lógica del negocio, la interfaz gráfica y el acceso a los datos, logrando un proyecto más organizado y fácil de mantener.

En MoneyTrack, el **Modelo** está representado principalmente por la clase DatabaseHelper y la estructura de la base de datos SQLite; la **Vista** corresponde a los archivos XML que conforman las interfaces de usuario; mientras que el **Controlador** está compuesto por las diferentes Activities encargadas de procesar las acciones del usuario y coordinar la comunicación entre la interfaz y la base de datos.

7-Arquitectura del sistema

Para el desarrollo de **MoneyTrack** se utilizó el patrón de arquitectura **Modelo–Vista–Controlador (MVC)**. Esta arquitectura fue seleccionada porque permite dividir la aplicación en componentes con responsabilidades diferentes, facilitando la organización del código, la comprensión del funcionamiento del sistema y su mantenimiento.

La separación por responsabilidades resulta especialmente útil en una aplicación como MoneyTrack, ya que existen elementos claramente diferenciados: las pantallas que utiliza el usuario, la lógica que procesa sus acciones y la base de datos donde se almacenan los registros. Al organizar estos elementos mediante MVC, se evita concentrar toda la lógica en un solo archivo y se facilita la incorporación de mejoras futuras.

Patrón utilizado: Modelo–Vista–Controlador

El patrón MVC divide el sistema en tres componentes principales: Modelo, Vista y Controlador.

Modelo

El Modelo representa los datos de la aplicación y las operaciones relacionadas con su almacenamiento. En MoneyTrack, este componente está representado principalmente por la clase DatabaseHelper, la cual extiende de SQLiteOpenHelper.

Esta clase se encarga de:

- Crear y actualizar la base de datos MoneyTrack.db.
- Definir las tablas del sistema.
- Registrar usuarios.
- Validar las credenciales de inicio de sesión.
- Verificar si un correo ya está registrado.
- Obtener el nombre del usuario.
- Insertar nuevos gastos.
- Consultar los gastos almacenados.
- Actualizar los datos de un gasto.
- Eliminar gastos.
- Calcular la cantidad de gastos registrados.
- Calcular el monto total gastado.

Aunque el modelo de datos contempla las entidades Usuario, Categoría, Ingreso y Gasto, la funcionalidad principal de esta versión utiliza principalmente las tablas Usuario y Gasto. Las demás entidades permanecen definidas como parte del diseño general y pueden utilizarse en futuras ampliaciones.

Vista

La Vista corresponde a las interfaces gráficas mediante las cuales el usuario interactúa con la aplicación. En MoneyTrack, las vistas fueron desarrolladas utilizando archivos XML ubicados dentro del directorio de recursos de Android.

Entre las principales vistas se encuentran:

- activity_main.xml: pantalla de inicio de sesión.
- activity_registro_usuario.xml: formulario para crear una cuenta.
- activity_dashboard.xml: panel principal con el resumen de gastos.
- activity_registrar_gasto.xml: formulario para guardar nuevos gastos.
- activity_lista_gastos.xml: pantalla para consultar, actualizar y eliminar gastos.

Los archivos XML definen elementos como campos de texto, botones, títulos, tarjetas, colores, márgenes y distribución de los componentes. Esto permite mantener separada la presentación visual de la lógica programada en Java.

También se utilizaron archivos dentro de la carpeta drawable para mantener un diseño coherente, como fondos redondeados para botones, campos y tarjetas.

Controlador

El Controlador recibe las acciones realizadas por el usuario, ejecuta las validaciones correspondientes y se comunica con el Modelo para consultar o modificar la información.

En MoneyTrack, esta responsabilidad es asumida principalmente por las Activities:

- MainActivity: procesa el inicio de sesión, valida los campos y permite navegar hacia el registro de usuario.
- RegistroUsuarioActivity: valida la información ingresada y registra nuevos usuarios en SQLite.
- DashboardActivity: muestra el nombre del usuario, la cantidad de gastos y el total gastado, además de controlar la navegación principal.
- RegistrarGastoActivity: procesa el formulario de registro de gastos y ejecuta las validaciones del monto y la descripción.
- ListaGastosActivity: consulta los gastos y gestiona las operaciones de actualización y eliminación.

Las Activities funcionan como intermediarias entre los archivos XML y la clase DatabaseHelper. Por ejemplo, cuando el usuario presiona el botón para guardar un gasto, RegistrarGastoActivity obtiene los datos escritos en la Vista, los valida y luego llama al método insertarGasto() del Modelo.

Capas del sistema

Aunque el proyecto utiliza el patrón MVC, su funcionamiento también puede explicarse mediante tres capas principales.

Capa de presentación

Esta capa contiene las pantallas y los componentes visuales que utiliza el usuario. Está formada por los archivos XML y por los elementos gráficos ubicados en la carpeta drawable.

Su responsabilidad es presentar la información, recibir datos mediante formularios y ofrecer una navegación clara entre las distintas funcionalidades.

Capa de lógica de aplicación

Esta capa está compuesta por las Activities desarrolladas en Java. Aquí se encuentran las validaciones, la navegación, la gestión de eventos de los botones y las decisiones relacionadas con el flujo de la aplicación.

Por ejemplo, esta capa verifica que el correo tenga un formato válido, que las contraseñas coincidan, que el monto sea mayor que cero y que el usuario confirme una eliminación antes de ejecutarla.

Capa de datos

La capa de datos está representada por SQLite y por la clase DatabaseHelper. Su responsabilidad es conservar la información de manera persistente y ejecutar las operaciones solicitadas por la capa de lógica.

Esta separación permite que las pantallas no ejecuten directamente instrucciones SQL, sino que utilicen métodos específicos definidos dentro de DatabaseHelper.

Estructura de carpetas

El proyecto se encuentra organizado dentro de la estructura estándar de una aplicación Android. Los elementos principales se distribuyen de la siguiente manera:

app

```
└─ src
    └─ main
        └─ java
            └─ com.example.moneytrack
                └─ MainActivity.java
```

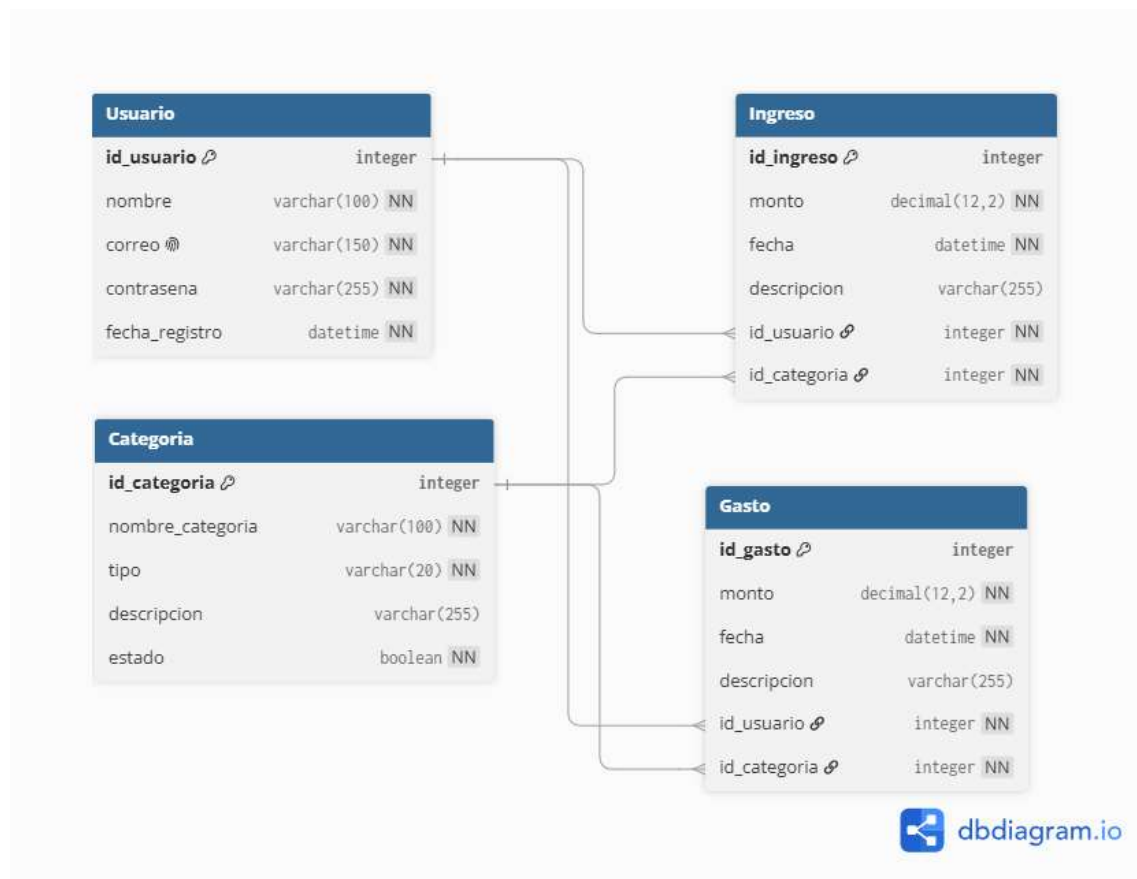
```
|   |— RegistroUsuarioActivity.java
|   |— DashboardActivity.java
|   |— RegistrarGastoActivity.java
|   |— ListaGastosActivity.java
|   |— DatabaseHelper.java
|
|— res
|   |— layout
|   |   |— activity_main.xml
|   |   |— activity_registro_usuario.xml
|   |   |— activity_dashboard.xml
|   |   |— activity_registrar_gasto.xml
|   |   |— activity_lista_gastos.xml
|   |
|   |— drawable
|   |   |— fondo_tarjeta.xml
|   |   |— fondo_campo.xml
|   |   |— fondo_boton_principal.xml
|   |   |— fondo_boton_secundario.xml
|   |   |— fondo_boton_eliminar.xml
|   |
|   |— values
|   |   |— colors.xml
|   |   |— strings.xml
|   |   |— themes.xml
|
|— AndroidManifest.xml
```

8-Modelo de datos

El modelo de datos de **MoneyTrack** fue diseñado para representar la información necesaria para la gestión de usuarios y movimientos financieros personales. El diseño general está compuesto por cuatro entidades principales: **Usuario**, **Categoría**, **Ingreso** y **Gasto**. Estas entidades permiten almacenar los datos de las personas registradas, clasificar los movimientos financieros y mantener un historial de los ingresos y gastos realizados.

Durante el diseño inicial se representó la base de datos mediante un **Diagrama Entidad-Relación (DER)**. Para la aplicación Android se utilizó SQLite como sistema de almacenamiento local. En la versión funcional desarrollada, las tablas utilizadas directamente son **Usuario** y **Gasto**, debido a que el alcance principal se concentró en la autenticación y en el CRUD completo de gastos. Las tablas **Categoría** e **Ingreso** fueron incluidas en la estructura general para mantener coherencia con el diseño original y facilitar futuras ampliaciones.

Diagrama Entidad-Relación final



Diccionario de datos

Tabla Usuario

La tabla Usuario almacena las cuentas creadas dentro de la aplicación.

Campo	Tipo de dato	Restricciones	Descripción
id_usuario	INTEGER	Clave primaria, autoincremental	Identificador único de cada usuario registrado.
nombre	TEXT	No nulo	Nombre completo del usuario.
correo	TEXT	No nulo, único	Correo utilizado para identificar al usuario e iniciar sesión.
contrasena	TEXT	No nulo	Contraseña utilizada para autenticar la cuenta.
fecha_registro	TEXT	No nulo	Fecha en la que se creó la cuenta del usuario.

El campo id_usuario se genera automáticamente al insertar un registro. La restricción UNIQUE aplicada sobre correo evita cuentas duplicadas. Antes de registrar un nuevo usuario, la aplicación consulta si el correo ya se encuentra almacenado y muestra un mensaje de validación cuando existe.

Tabla Categoria

La tabla Categoria contiene la clasificación definida para los movimientos financieros.

Campo	Tipo de dato	Restricciones	Descripción
id_categoria	INTEGER	Clave primaria, autoincremental	Identificador único de la categoría.
nombre_categoria	TEXT	No nulo	Nombre asignado a la categoría.
tipo	TEXT	No nulo	Indica si la categoría corresponde a un ingreso o a un gasto.
descripcion	TEXT	Opcional	Información adicional acerca de la categoría.
estado	INTEGER	No nulo	Indica si la categoría está activa o inactiva.

El campo estado se almacena como un valor entero debido a que SQLite no posee un tipo booleano independiente. Convencionalmente, el valor 1 puede representar una categoría activa y el valor 0 una categoría inactiva.

Tabla Ingreso

La tabla Ingreso fue diseñada para almacenar las entradas de dinero del usuario.

Campo	Tipo de dato	Restricciones	Descripción
id_ingreso	INTEGER	Clave primaria, autoincremental	Identificador único del ingreso.
monto	REAL	No nulo	Cantidad de dinero recibida.
fecha	TEXT	No nulo	Fecha en la que se registró o recibió el ingreso.
descripcion	TEXT	Opcional	Información complementaria sobre el ingreso.
id_usuario	INTEGER	Clave foránea	Identifica al usuario propietario del ingreso.
id_categoria	INTEGER	Clave foránea	Identifica la categoría asignada al ingreso.

El campo id_usuario se relaciona con la clave primaria de la tabla Usuario, mientras que id_categoria se relaciona con la tabla Categoria. De esta manera, cada ingreso puede vincularse con el usuario que lo registró y con su clasificación correspondiente.

Tabla Gasto

La tabla Gasto almacena las salidas de dinero registradas desde la aplicación.

Campo	Tipo de dato	Restricciones	Descripción
id_gasto	INTEGER	Clave primaria, autoincremental	Identificador único del gasto.
monto	REAL	No nulo	Cantidad de dinero correspondiente al gasto.
fecha	TEXT	No nulo	Fecha en la que se registró el gasto.
descripcion	TEXT	Opcional	Detalle o motivo del gasto realizado.
id_usuario	INTEGER	Clave foránea, opcional en esta versión	Identifica al usuario relacionado con el gasto.
id_categoria	INTEGER	Clave foránea, opcional en esta versión	Identifica la categoría asignada al gasto.

En la versión actual, el formulario solicita el monto y una descripción, mientras que la fecha se genera automáticamente utilizando la fecha del dispositivo. Las columnas id_usuario e id_categoria forman parte de la estructura prevista en el modelo, pero todavía no se asignan al guardar los gastos. Esto se dejó como una mejora para una versión futura, en la que cada usuario podrá consultar exclusivamente sus propios movimientos y clasificarlos por categorías.

9-Descripción de pantallas

Pantalla de inicio de sesión

La pantalla de inicio de sesión constituye el punto de acceso principal a la aplicación. Su propósito es autenticar al usuario mediante el ingreso de su correo electrónico y contraseña, garantizando que únicamente los usuarios registrados puedan acceder a las funcionalidades del sistema.

Al presionar el botón **"Iniciar sesión"**, la aplicación valida que las credenciales coincidan con la información almacenada en la base de datos SQLite. Si la autenticación es correcta, el usuario es dirigido al Dashboard; en caso contrario, se muestra un mensaje indicando que el correo o la contraseña son incorrectos. Desde esta misma pantalla también es posible acceder al formulario de registro mediante el botón **"Crear cuenta"**.



Pantalla de registro de usuario

La pantalla de registro permite crear nuevas cuentas dentro de la aplicación. En ella, el usuario debe ingresar su nombre, correo electrónico y contraseña para completar el proceso de registro.

Antes de almacenar la información, la aplicación verifica que todos los campos obligatorios hayan sido completados y que el correo electrónico no se encuentre registrado previamente. Si las validaciones son satisfactorias, la información se guarda en la base de datos SQLite y se muestra un mensaje confirmando que el registro fue realizado correctamente.



The screenshot shows a mobile application interface for creating a new account. At the top, the status bar displays the time 4:22, signal strength, Wi-Fi, and battery level at 67%. The main heading is 'Crear cuenta' in blue. Below it, a subtitle reads 'Complete los datos para comenzar a utilizar MoneyTrack'. The form consists of four text input fields: 'Nombre completo', 'Correo electrónico', 'Contraseña', and 'Confirmar contraseña'. At the bottom of the form is a purple button labeled 'Crear cuenta'. The bottom of the screen shows the standard Android navigation bar with back, home, and recent apps icons.

Pantalla Dashboard

El Dashboard representa la pantalla principal de la aplicación después de iniciar sesión. Su objetivo es ofrecer un resumen general de la información almacenada y servir como punto de navegación hacia las demás funcionalidades.

En esta pantalla se muestra un mensaje de bienvenida personalizado con el nombre del usuario, la cantidad de gastos registrados y el monto total gastado hasta el momento. Además, incorpora botones para registrar nuevos gastos, consultar la lista de gastos y cerrar la sesión de forma segura.



Pantalla Registrar gasto

La pantalla de registro de gastos permite ingresar nuevos movimientos financieros dentro de la aplicación. El usuario introduce el monto del gasto y una descripción que facilite su identificación.

Al guardar la información, la aplicación valida que el monto haya sido ingresado correctamente y sea mayor que cero. Posteriormente, genera automáticamente la fecha del registro y almacena toda la información en la base de datos SQLite. Si el proceso se completa correctamente, se muestra un mensaje confirmando que el gasto fue registrado exitosamente.



Pantalla Lista de gastos

La pantalla de lista de gastos permite consultar todos los gastos registrados en la aplicación. La información se presenta mostrando el identificador del gasto, el monto, la fecha y la descripción correspondiente.

Desde esta misma pantalla el usuario puede actualizar la información de un gasto existente o eliminar un registro cuando ya no sea necesario. Antes de realizar estas operaciones, la aplicación valida la información ingresada y, una vez completada la acción, actualiza automáticamente la lista para reflejar los cambios realizados en la base de datos.



4:24 67%

Gastos registrados

No hay gastos registrados.

Actualizar gasto

ID del gasto

Nuevo monto

Nueva descripción

Actualizar gasto

Eliminar gasto

ID del gasto

Eliminar gasto

10-Decisiones de diseño

Durante el desarrollo de **MoneyTrack** se tomaron distintas decisiones técnicas y funcionales con el propósito de mantener una aplicación sencilla, estable y coherente con el alcance del proyecto. Estas decisiones se basaron en los requisitos de la asignatura, el tiempo disponible, las tecnologías dominadas y la necesidad de entregar una aplicación móvil completamente funcional.

Uso de Android Studio como entorno de desarrollo

Se decidió utilizar **Android Studio** como entorno principal porque es la herramienta oficial para el desarrollo de aplicaciones Android. Esta elección facilitó la creación de Activities, el diseño de interfaces, la compilación del proyecto, la ejecución en emuladores y la generación del archivo APK.

Se descartó continuar con otras alternativas como .NET MAUI o editores más generales, debido a que presentaban mayor complejidad para el alcance del proyecto y podían aumentar el tiempo de desarrollo. Android Studio ofrecía una integración más directa con Java, XML y SQLite.

Uso de Java como lenguaje principal

Se eligió **Java** como lenguaje de programación porque era el lenguaje trabajado durante la asignatura y porque ofrece compatibilidad completa con el desarrollo Android nativo.

Esta decisión permitió implementar la lógica de autenticación, validaciones, navegación y operaciones CRUD sin incorporar tecnologías nuevas que pudieran complicar el proyecto. Se descartó utilizar Kotlin en esta versión, aunque podría considerarse en una futura actualización.

Uso de SQLite como base de datos local

Se seleccionó **SQLite** como sistema de persistencia porque permite almacenar información directamente en el dispositivo sin depender de conexión a Internet ni de servidores externos.

Esta decisión fue adecuada para la naturaleza de MoneyTrack, ya que la aplicación debía registrar usuarios y gastos de forma local. También permitió cumplir con el requisito de persistencia y con el CRUD completo solicitado en la consigna.

Se descartaron servicios como Firebase, Supabase o una conexión directa con SQL Server, debido a que requerían configuración adicional, conectividad y, en algunos casos, una API intermedia. También se descartó Room para mantener la implementación alineada con SQLite y con las tecnologías ya utilizadas durante el desarrollo.

Uso del patrón MVC

Se decidió utilizar el patrón **Modelo–Vista–Controlador (MVC)** para separar la interfaz, la lógica y el acceso a los datos.

Los archivos XML representan la Vista, las Activities cumplen la función de Controlador y la clase DatabaseHelper junto con SQLite representan el Modelo. Esta organización permitió mantener una estructura comprensible y facilitó la corrección de errores durante el desarrollo.

Se descartaron arquitecturas más complejas como MVVM o Clean Architecture, ya que para el alcance académico de MoneyTrack podían añadir una complejidad innecesaria.

Priorización del CRUD de gastos

Se definió el CRUD de **Gasto** como la funcionalidad principal del proyecto. Esta decisión se tomó porque la consigna exige al menos un conjunto completo de operaciones CRUD y porque el registro de gastos constituye el núcleo de MoneyTrack.

Se implementaron las operaciones de crear, consultar, actualizar y eliminar gastos. También se añadieron validaciones y confirmación antes de eliminar.

Se descartó desarrollar un CRUD completo de ingresos y categorías dentro de esta versión, ya que habría ampliado demasiado el alcance y aumentado el riesgo de no completar correctamente la funcionalidad principal.

Simplificación del formulario de gastos

Inicialmente se contempló incluir un campo de categoría en el formulario de gastos. Sin embargo, se decidió eliminarlo para mantener el flujo más simple y evitar inconsistencias entre la interfaz y la base de datos.

El formulario final solicita únicamente monto y descripción, mientras que la fecha se genera automáticamente. Esta decisión redujo la cantidad de campos obligatorios y permitió concentrarse en el funcionamiento estable del CRUD.

La gestión de categorías quedó planteada como una mejora futura.

Registro e inicio de sesión local

Se decidió incorporar registro e inicio de sesión utilizando la tabla Usuario de SQLite. Esto permitió que la aplicación tuviera un flujo más completo y que la tabla de usuarios fuera utilizada realmente.

También se añadió la restricción de correo único y validación de credenciales. Se descartó una autenticación remota porque no era necesaria para esta versión y habría requerido servicios externos.

Dashboard con datos reales

El Dashboard fue diseñado para mostrar información obtenida directamente de SQLite, como la cantidad de gastos registrados y el total gastado.

Se decidió evitar un Dashboard únicamente decorativo, ya que mostrar datos reales mejora la utilidad de la aplicación y demuestra la integración entre la interfaz y la base de datos.

Se descartaron gráficos, reportes visuales y estadísticas avanzadas porque no eran indispensables para cumplir con la funcionalidad principal.

Diseño visual coherente

Se decidió mantener una identidad visual uniforme en todas las pantallas, utilizando colores consistentes, fondos claros, tarjetas, campos redondeados y botones con estilos similares.

Esta decisión se tomó para mejorar la experiencia del usuario y cumplir con el criterio de diseño móvil de la rúbrica.

Se descartó utilizar librerías externas de diseño o animaciones avanzadas para evitar dependencias innecesarias y conservar la estabilidad del proyecto.

Actualización y eliminación por ID

La pantalla de lista permite actualizar y eliminar gastos utilizando el identificador del registro. Esta solución fue elegida porque era sencilla de implementar y permitía completar el CRUD de forma estable.

Se descartó, por el momento, utilizar un RecyclerView con botones individuales o una pantalla de detalle por cada gasto, ya que eso implicaba modificar considerablemente la estructura actual. Esta mejora podría incorporarse en una versión futura para facilitar la interacción.

Almacenamiento local y funcionamiento sin Internet

La aplicación fue diseñada para funcionar completamente sin conexión a Internet. Esta decisión mejora la accesibilidad y evita que el usuario dependa de servicios externos.

Como consecuencia, los datos permanecen en el dispositivo donde se instala la aplicación. Se descartó la sincronización entre dispositivos, las copias de

seguridad automáticas y el almacenamiento en la nube para mantener el alcance controlado.

Seguridad de la contraseña

Para esta versión académica, la contraseña se almacena directamente en la base de datos local para permitir la validación del inicio de sesión.

Esta decisión simplificó la implementación, pero se reconoce que en una aplicación destinada a producción sería necesario aplicar técnicas de seguridad adicionales, como cifrado o almacenamiento mediante funciones hash. Esa mejora quedó fuera del alcance de esta versión.

Conclusión de las decisiones de diseño

Las decisiones tomadas permitieron construir una aplicación funcional, comprensible y adecuada al tiempo disponible. En lugar de ampliar el alcance con múltiples módulos incompletos, se priorizó una funcionalidad central estable, un flujo completo de usuario, persistencia local, validaciones y una interfaz coherente.

Las funcionalidades descartadas no se consideran eliminadas definitivamente, sino posibles mejoras para futuras versiones de MoneyTrack.

11-Plan y resultados de pruebas

Con el objetivo de garantizar el correcto funcionamiento de la aplicación **MoneyTrack**, se diseñó un plan de pruebas enfocado en verificar las funcionalidades principales implementadas durante el desarrollo. Las pruebas fueron ejecutadas de forma progresiva a medida que se completaban los diferentes módulos de la aplicación, permitiendo identificar errores de funcionamiento y corregirlos antes de la integración final.

Las pruebas se realizaron utilizando el emulador de Android Studio y un dispositivo Android, comprobando tanto el comportamiento de la interfaz como la interacción con la base de datos SQLite. Se evaluaron los procesos de autenticación, registro de usuarios, navegación entre pantallas y las operaciones CRUD sobre los gastos registrados.

Plan de pruebas

Las pruebas funcionales se planificaron considerando los procesos más importantes que realiza un usuario dentro de la aplicación. Entre los aspectos evaluados se encuentran:

- Registro de nuevos usuarios.
- Inicio de sesión con credenciales válidas e inválidas.
- Validación de campos obligatorios en los formularios.
- Registro de nuevos gastos.
- Consulta de la lista de gastos almacenados.
- Actualización de la información de un gasto.
- Eliminación de gastos registrados.
- Actualización automática del Dashboard después de cada operación.
- Navegación entre todas las pantallas de la aplicación.
- Persistencia de la información almacenada en SQLite.

Cada prueba fue ejecutada varias veces para verificar que el comportamiento fuera consistente y que la aplicación mantuviera la integridad de la información almacenada.

Resultados obtenidos

Los resultados obtenidos fueron satisfactorios, ya que todas las funcionalidades principales de MoneyTrack respondieron correctamente después de aplicar las correcciones realizadas durante el desarrollo.

Se verificó que el sistema registra correctamente los nuevos usuarios, valida el inicio de sesión utilizando la base de datos SQLite y restringe el acceso cuando las credenciales no son válidas. Asimismo, el CRUD de gastos permite crear, consultar, actualizar y eliminar registros de manera correcta, manteniendo la información almacenada incluso después de cerrar la aplicación.

El Dashboard también fue validado, comprobando que muestra correctamente el nombre del usuario autenticado, la cantidad de gastos registrados y el monto total gastado, actualizando esta información conforme se realizan nuevas operaciones.

En términos generales, la aplicación presentó un comportamiento estable y cumplió con los objetivos funcionales definidos para esta versión.

Errores encontrados y correcciones aplicadas

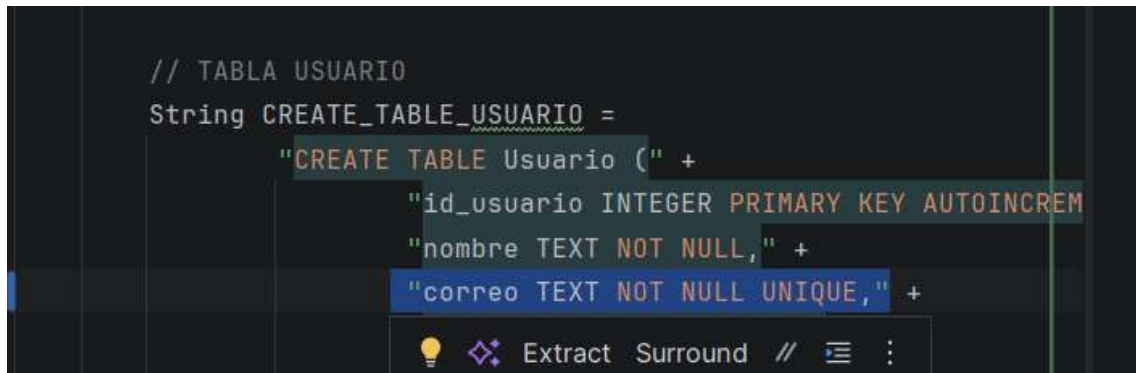
Durante el desarrollo del proyecto se identificaron diversos inconvenientes técnicos que fueron corregidos antes de finalizar la aplicación. Los principales fueron los siguientes:

Error en el registro de usuarios

Inicialmente la aplicación permitía registrar múltiples usuarios utilizando el mismo correo electrónico.

Corrección aplicada: Se agregó una restricción **UNIQUE** al campo correo en la tabla **Usuario** y se implementó una validación previa para impedir registros duplicados.

```
// TABLA USUARIO
String CREATE_TABLE_USUARIO =
    "CREATE TABLE Usuario (" +
        "id_usuario INTEGER PRIMARY KEY AUTOINCREM" +
        "nombre TEXT NOT NULL," +
        "correo TEXT NOT NULL UNIQUE," +
```

A screenshot of a code editor showing SQL code for creating a table named 'Usuario'. The code is in a dark-themed editor with syntax highlighting. The code defines a table with three columns: 'id_usuario' as an integer primary key with auto-increment, 'nombre' as a text field that is not null, and 'correo' as a text field that is not null and has a unique constraint. The code is wrapped in a string variable named 'CREATE_TABLE_USUARIO'. At the bottom of the editor, there is a toolbar with icons for a lightbulb, a magnifying glass, and buttons for 'Extract', 'Surround', and a comment icon.

Error en el inicio de sesión

En una versión inicial, el botón **Iniciar sesión** permitía acceder al Dashboard sin verificar las credenciales ingresadas.

Corrección aplicada: Se implementó el método validarUsuario() en DatabaseHelper, permitiendo comprobar el correo y la contraseña almacenados en SQLite antes de conceder el acceso.


```

public boolean validarUsuario(String correo, String contraseña) {
    SQLiteDatabase db = this.getReadableDatabase();

    Cursor cursor = db.rawQuery(
        "SELECT * FROM Usuario WHERE correo = ? AND contraseña = ?",
        new String[]{correo, contraseña}
    );

    boolean existe = cursor.getCount() > 0;

    cursor.close();

    return existe;
}

```

Error durante el registro de gastos

Al principio, el formulario permitía intentar registrar gastos sin completar correctamente la información requerida.

Corrección aplicada: Se añadieron validaciones para comprobar que el monto fuera obligatorio y mayor que cero antes de guardar el registro.

```

if (montoTexto.isEmpty()) {
    etMonto.setError("Debe ingresar un monto");
    etMonto.requestFocus();
    return;
}

double monto;

try {
    monto = Double.parseDouble(montoTexto);
} catch (NumberFormatException e) {
    etMonto.setError("Ingresa un monto válido");
    etMonto.requestFocus();
    return;
}

if (monto <= 0) {
    etMonto.setError("El monto debe ser mayor que cero");
    etMonto.requestFocus();
    return;
}

```

Error de actualización y eliminación

Durante las primeras pruebas se detectaron inconvenientes relacionados con la actualización y eliminación de registros cuando el identificador ingresado no existía.

Corrección aplicada: Se incorporaron validaciones para verificar la existencia del registro y mostrar mensajes informativos al usuario cuando el ID no era válido.

```
if (idTexto.isEmpty()) {
    etIdEliminar.setError("Ingrese el ID del gasto");
    etIdEliminar.requestFocus();
    return;
}

int idGasto;

try {
    idGasto = Integer.parseInt(idTexto);
} catch (NumberFormatException e) {
    etIdEliminar.setError("Ingrese un ID válido");
    etIdEliminar.requestFocus();
    return;
}

if (idGasto <= 0) {
    etIdEliminar.setError("El ID debe ser mayor que cero");
    etIdEliminar.requestFocus();
    return;
}

mostrarConfirmacionEliminacion(idGasto);
}
```

```
if (eliminado) {  
  
    Toast.makeText(  
        context: this,  
        text: "Gasto eliminado correctamente",  
        Toast.LENGTH_SHORT  
    ).show();  
  
    etIdEliminar.setText("");  
  
    mostrarGastos();  
  
} else {  
  
    Toast.makeText(  
        context: this,  
        text: "No existe un gasto con ese ID",  
        Toast.LENGTH_SHORT  
    ).show();  
  
}  
  
}
```

Error en la navegación entre pantallas

Se presentaron algunos problemas relacionados con la creación de nuevas Activities y la navegación dentro de la aplicación.

Corrección aplicada: Se ajustó el archivo AndroidManifest.xml y se verificó la correcta comunicación mediante Intent entre todas las pantallas implementadas.

```
btnRegistrarGasto.setOnClickListener( View v -> {
    Intent intent = new Intent(
        packageContext: DashboardActivity.this,
        RegistrarGastoActivity.class
    );
    startActivity(intent);
});

btnVerGastos.setOnClickListener( View v -> {
    Intent intent = new Intent(
        packageContext: DashboardActivity.this,
        ListaGastosActivity.class
    );
    startActivity(intent);
});

btnCerrarSesion.setOnClickListener( View v -> {

    Intent intent = new Intent(
        packageContext: DashboardActivity.this,
        MainActivity.class
    );

    intent.setFlags(
        Intent.FLAG_ACTIVITY_NEW_TASK |
        Intent.FLAG_ACTIVITY_CLEAR_TASK
    );

    startActivity(intent);
});
```

Error en la creación de actividades

Durante la implementación de la pantalla de registro aparecieron errores relacionados con la plantilla generada automáticamente por Android Studio, específicamente con el identificador main y componentes de Edge-to-Edge que no existían en el diseño.

Corrección aplicada: Se eliminó el código innecesario generado automáticamente y se simplificó la Activity para adaptarla al diseño utilizado en el proyecto.

Error de compilación

En distintas etapas del desarrollo surgieron errores ocasionados por declaraciones duplicadas de package e import, así como por configuraciones propias del proyecto.

Corrección aplicada: Se reorganizaron las importaciones, se eliminaron declaraciones repetidas y se verificó la estructura de cada archivo antes de continuar con el desarrollo.

Conclusión de las pruebas

Las pruebas realizadas permitieron comprobar que **MoneyTrack** cumple correctamente con las funcionalidades previstas para esta primera versión. La aplicación presentó un funcionamiento estable durante las pruebas finales, manteniendo la integridad de la información almacenada y ofreciendo una navegación fluida entre las diferentes pantallas.

Los errores detectados durante el desarrollo fueron corregidos oportunamente, lo que permitió entregar una aplicación funcional con autenticación de usuarios, persistencia local mediante SQLite y un CRUD completo para la gestión de gastos personales. Los casos de prueba detallados y sus evidencias se presentan en el apartado correspondiente de la documentación.

Conclusion:

Con MoneyTrack se cumplió el objetivo principal del proyecto. Diseñar e implementar una app móvil para Android que registre y gestione gastos personales, guardando todo en SQLite. El resultado fue una app funcional. Tiene sistema de registro e inicio de sesión, un Dashboard con información resumida, un CRUD completo para manejar gastos. Cumple con lo que pedía el proyecto práctico final.

Durante el desarrollo pude aplicar lo aprendido en la carrera. Programación orientada a objetos, desarrollo de apps móviles, bases de datos relacionales, diseño de interfaces, arquitectura de software. El proyecto también me ayudó a fortalecer otras habilidades. Resolver problemas, depurar código, implementar validaciones, usar herramientas de control de versiones, escribir documentación técnica.

Aprendí que planificar bien antes de programar es fundamental. Definir el modelo de datos, la arquitectura del sistema, el alcance del proyecto. Esto mantuvo todo organizado y facilitó meter nuevas funcionalidades sobre la marcha. Las pruebas en cada etapa ayudaron mucho. Se detectaban y corregían errores a tiempo, mejorando la estabilidad de la app.

Si el proyecto siguiera creciendo, le metería más funcionalidades. Gestión completa de ingresos y categorías, reportes y gráficos estadísticos, manejo de presupuestos, exportar información, sincronizar datos con la nube. Todo esto ampliaría lo que MoneyTrack puede hacer, daría una experiencia más completa al usuario.

MoneyTrack es una solución funcional para registrar y controlar gastos personales. Muestra también la aplicación práctica de lo aprendido en la carrera de Ingeniería de Software. El proyecto unió análisis, diseño, desarrollo, pruebas y documentación en un mismo proceso. El resultado cumple los objetivos planteados, y deja una base sólida para futuras mejoras.

Bibliografía:

Develop for Android. (s. f.-b). Android Developers. <https://developer.android.com/develop>

Learn Java - dev.java. (s. f.). Dev.java: The Destination For Java Developers.

<https://dev.java/learn/>

SQLite Documentation. (s. f.-b). <https://sqlite.org/docs.html>

GitHub Documentación de ayuda de .com. (s. f.). GitHub Docs. <https://docs.github.com/es>